

K L UNIVERSITY
FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab

Report On

Smart Parking Allocation and Occupancy

SUBMITTED BY:

ID NUMBER	NAME
2500032415	P. Aman khan

UNDER THE ESTEEMED GUIDANCE

OF Dr. SREERAM

ASSISTANT PROFESSOR



KL UNIVERSITY
Green fields, Vaddeswaram – 522 502
Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCES



CERTIFICATE

This is to certify that the project based laboratory report entitled “SMART PARKING ALLOCATION AND OCCUPANCY” submitted by **P. AMAN KHAN** bearing Regd. No.2500032415 to the **Department of COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in “Python Programming (25SC2007)” course in I B Tech II Semester, is a Bonafede record of the work carried out by him/her under my supervision during the academic year 2025– 2026.

PROJECT SUPERVISOR

Dr. SREERAM

HEAD OF THE DEPARTMENT

Dr. D. HARITHA

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President.

Sri. Koneru Satyanarayana, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth.

I express sincere gratitude to our Coordinator and HOD-BES **Dr. D. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. SREERAM** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge our indebtedness to all those who devoted themselves directly or indirectly to making this project a success.

NAME: P. AMAN KHAN

ID No: 2500032415

ABSTRACT

This project is aimed at developing a **Smart Parking Allocation and Occupancy System** that can manage vehicle parking in a more organized, efficient, and intelligent manner. In daily life, parking has become a major concern in cities, colleges, shopping malls, hospitals, office complexes, and residential areas. Drivers often spend a long time searching for an empty slot, which causes traffic congestion, waste of fuel, frustration, and poor use of available space. This project is designed to reduce those problems by showing parking availability clearly and assigning slots in an automatic or semi-automatic way.

The system monitors parking spaces and identifies whether a slot is occupied or free. Based on this information, it helps in allocating the correct slot to an incoming vehicle without AI confusion. It can also maintain vehicle entry and exit records, slot status, occupancy details, and parking duration. By doing so, the system becomes useful not only for users but also for parking administrators who need accurate information to manage space properly. In a real-world setup, this project can be linked with sensors, cameras, or a software database to provide live updates. In a simple implementation, it can be developed using a program that checks availability and allocates slots logically.

The main objective of this project is to reduce manual work, save time, and improve the parking experience. It helps avoid unnecessary crowding at parking entrances and allows vehicles to move in a smoother flow. It also improves the overall utilization of parking space by ensuring that empty slots do not remain unused due to poor management. This system is especially helpful in busy public places where parking demand is high and space is limited.

In future, this project can be enhanced with mobile application support, digital entry passes, QR code scanning, automated billing, real-time slot display, and notification alerts. These upgrades can make the parking process even faster and smarter.

Key Words: smart parking, occupancy detection, slot allocation, vehicle management, real-time availability, parking automation.

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	6
2	Aim of the Project	7
2.1	Advantages & Disadvantages	8
2.2	Future Implementation	8
3	Software & Hardware Details	9
4	Data Flow Diagram	10-11
5	Algorithm for each module	12
6	Implementation	13-15
7	Integration and System Testing	16-18
8	Conclusion	19

INTRODUCTION

Parking is one of the most common urban problems in the modern world. As the number of vehicles continues to grow, finding a suitable parking place has become difficult in almost every crowded location. Many drivers arrive at a destination only to spend several minutes, or even longer, searching for an available space. This creates a chain reaction of problems such as traffic jams near parking entrances, unnecessary fuel consumption, time loss, and irritation for users. In large places like shopping centers, airports, hospitals, universities, and corporate buildings, parking control becomes even more important because improper parking management can disturb the entire flow of movement.

A smart parking allocation and occupancy system is a solution to this problem. It is designed to detect available parking spaces, identify occupied slots, and provide accurate parking information to the user or administrator. The system can work using sensors, software logic, or a combination of both. When a vehicle enters the parking area, the system checks the current slot status and assigns a free parking space if one is available. If all spaces are full, the system informs the user immediately. This reduces manual checking and makes parking much more efficient.

The system is not only about assigning spaces but also about improving the overall parking process. It can record entry time, exit time, occupancy duration, and usage patterns. These details are useful for planning and analysis. For example, administrators can understand which time periods are busiest and how parking spaces are being used. This helps in making smarter decisions for future expansion or redesign.

This project is an important step toward modern transport management because it combines convenience, speed, and organization. It saves time for drivers, reduces pressure on parking staff, and supports better use of space. For these reasons, smart parking systems are becoming more useful in today's fast-moving world.

AIM

The main aim of this project is to develop a **Smart Parking Allocation and Occupancy System** that helps in managing parking spaces efficiently. The system is designed to identify available parking slots and allocate them to vehicles in a systematic way, reducing the time spent searching for parking.

This project aims to minimize traffic congestion inside parking areas by providing clear information about free and occupied slots. It also reduces manual work by automating the process of parking allocation and occupancy tracking.

Another objective of the system is to improve the utilization of available parking space and maintain records of vehicle entry and exit. This helps in better management and organization of parking areas.

Overall, the aim is to create a simple, user-friendly, and efficient parking system that saves time, reduces confusion, and improves the parking experience.

The project aims:

- To reduce the time spent by drivers searching for parking.
- To provide automatic slot allocation based on availability.
- To monitor parking occupancy accurately.
- To minimize manual work in parking management.
- To improve traffic flow inside parking areas.
- To make parking convenient, organized, and user friendly.
- To maintain records of vehicle entry, exit, and slot usage.

This system is useful in places where parking is crowded and needs better control. It helps both users and administrators by providing a faster and more reliable method of parking management.

ADVANTAGES

This system has many advantages that make parking simpler and smarter.

- It saves time by helping drivers find free slots quickly.
- It reduces traffic congestion inside and outside parking areas.
- It improves space utilization by assigning vehicles properly.
- It reduces manual errors in parking management.
- It provides real-time occupancy information.
- It helps in maintaining proper records of parked vehicles.
- It can be expanded in future with automation and payment features.
- It gives a better user experience in crowded places.

DISADVANTAGES

Along with its benefits, the system also has some limitations.

- It may require sensors, devices, or software setup costs.
- It depends on accurate detection of vehicle presence.
- It may fail if sensors or system components stop working.
- It needs regular maintenance and monitoring.
- It may require internet or network connectivity in advanced versions.
- It can be difficult to install in very large parking areas without proper planning.

FUTURE IMPLEMENTATION

This project can be improved further in many ways.

In future, the system can be connected with a mobile application so users can check parking availability before reaching the location. It can also include automatic ticket generation, QR code scanning, and cashless payment options. Smart cameras and AI-based vehicle detection can be added for better accuracy. A notification system can be included to inform users about available slots, parking expiry, or payment details. The system can also be integrated with cloud storage to keep records safely and allow remote monitoring.

SYSTEM REQUIREMENTS

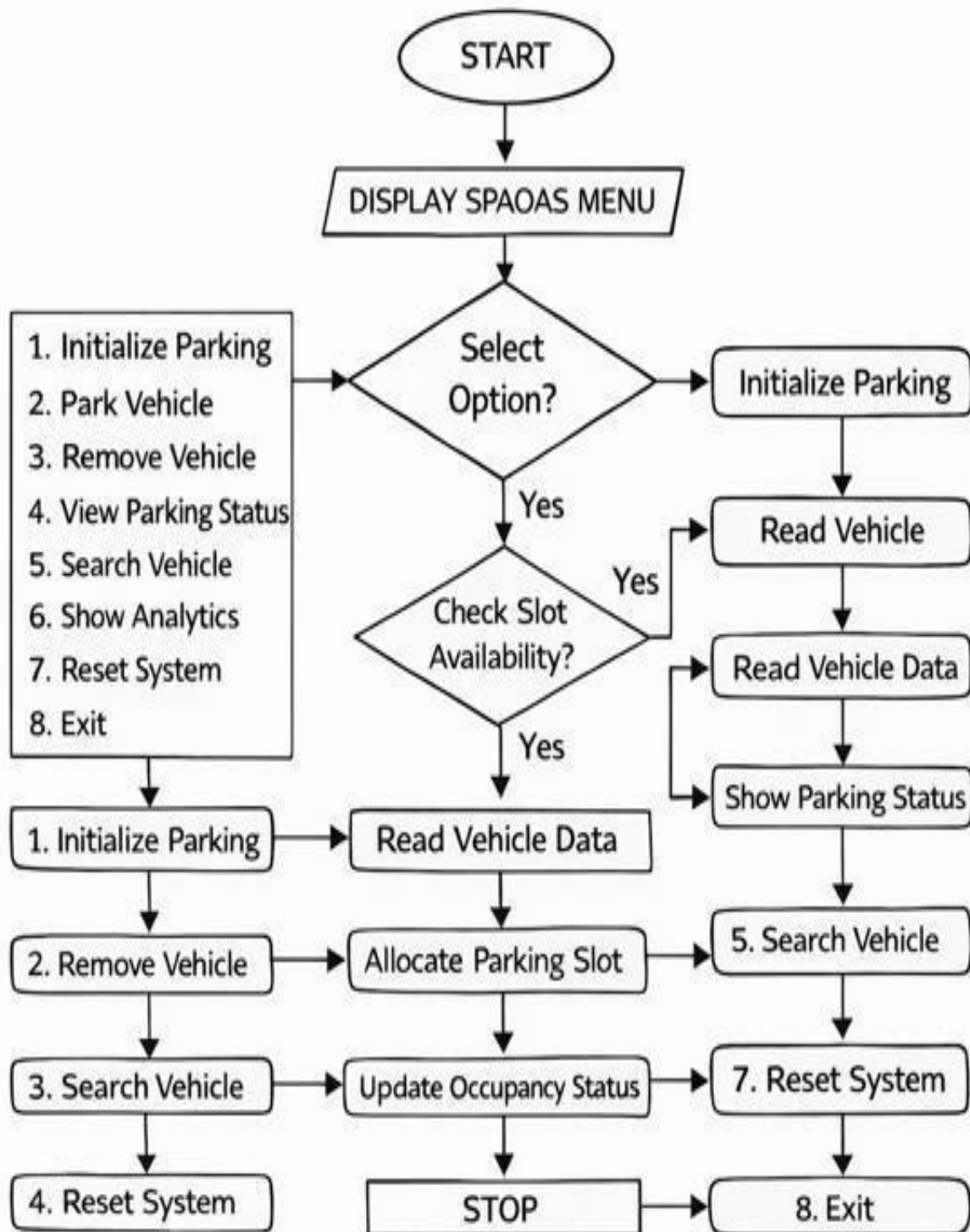
➤ SOFTWARE REQUIREMENTS:

- The major software requirements of the project are as follows:
- Language: Python / C / JavaScript depending on implementation.
Operating System: Windows 10 or above.
Database: MySQL / SQLite if record storage is required.
Tools: IDE such as VS Code, PyCharm, or Turbo C depending on the language.
Libraries: Sensor or GUI libraries if used in advanced version.

➤ HARDWARE REQUIREMENTS:

- The hardware requirements that support the software are as follows:
- RAM: 4 GB or above.
Processor: i3/i5 or above.
Input Devices: Keyboard, mouse, and if needed sensors.
Output Devices: Monitor and display unit.
Storage: Sufficient memory for records and application files.
- If implemented as a physical smart parking model, sensors, LEDs, microcontroller, and connecting wires may also be required

DATA FLOW DIAGRAM



ALGORITHM

Step 1: Start the program.

Step 2: Display the menu with options (1–8).

Step 3: Read user choice.

Step 4: If choice == 1 → call Initialize Parking.

Step 5: Else if choice == 2 → call Park Vehicle.

Step 6: Else if choice == 3 → call Remove Vehicle.

Step 7: Else if choice == 4 → call View Parking Status.

Step 8: Else if choice == 5 → call Search Vehicle.

Step 9: Else if choice == 6 → call Show Analytics.

Step 10: Else if choice == 7 → call Reset System.

Step 11: Else if choice == 8 → Exit program.

Step 12: Else → Display “Invalid Choice”.

Step 13: Repeat menu until user exits.

Step 14: Stop.

IMPLEMENTATION

```
import json
import datetime
import os
import pandas as pd
import matplotlib.pyplot as plt

FILE_NAME = "parking_data.json"
CSV_FILE = "parking_log.csv"
RATE_PER_HOUR = 20

# ----- CLASS (OOP) ----- #
class ParkingSystem:

    def __init__(self):
        self.data = self.load_data()

    # ----- FILE HANDLING ----- #
    def load_data(self):
        if not os.path.exists(FILE_NAME):
            return {}
        with open(FILE_NAME, "r") as file:
            return json.load(file)

    def save_data(self):
        with open(FILE_NAME, "w") as file:
            json.dump(self.data, file, indent=4)

    # ----- INITIALIZE ----- #
    def initialize(self):
        if self.data:
            print("Already initialized!")
            return

        try:
            slots = int(input("Enter total slots: "))
            self.data = {f"SLOT_{i}": None for i in range(1, slots + 1)} # dict + comprehension
            self.save_data()
```

```

    print("Parking initialized!")
except ValueError:
    print("Invalid input!")

# ----- PARK VEHICLE ----- #
def park(self):
    for slot in self.data:
        if self.data[slot] is None:
            vehicle = input("Vehicle number: ").strip().upper()
            owner = input("Owner name: ").strip().title()

            self.data[slot] = {
                "vehicle": vehicle,
                "owner": owner,
                "entry": datetime.datetime.now().isoformat()
            }

            self.save_data()
            print(f"Parked in {slot}")
            return

    print("Parking Full!")

# ----- REMOVE VEHICLE ----- #
def remove(self):
    vehicle = input("Enter vehicle number: ").strip().upper()

    for slot, info in self.data.items():
        if info and info["vehicle"] == vehicle:

            entry = datetime.datetime.fromisoformat(info["entry"])
            exit_time = datetime.datetime.now()

            duration = (exit_time - entry).total_seconds() / 3600
            fee = round(duration * RATE_PER_HOUR, 2)

            print(f"\nSlot: {slot}")
            print(f"Owner: {info['owner']}")
            print(f"Fee: ₹{fee}")

            # Save to CSV (Data pipeline concept)
            self.save_to_csv(vehicle, info['owner'], duration, fee)

```

```

        self.data[slot] = None
        self.save_data()
        return

    print("Vehicle not found!")

# ----- SAVE CSV ----- #
def save_to_csv(self, vehicle, owner, duration, fee):
    df = pd.DataFrame([[vehicle, owner, duration, fee]],
                      columns=["Vehicle", "Owner", "Hours", "Fee"])

    if not os.path.exists(CSV_FILE):
        df.to_csv(CSV_FILE, index=False)
    else:
        df.to_csv(CSV_FILE, mode='a', header=False, index=False)

# ----- SEARCH ----- #
def search(self):
    vehicle = input("Search vehicle: ").upper()

    found = [(slot, info) for slot, info in self.data.items()
             if info and info["vehicle"] == vehicle]

    if found:
        slot, info = found[0]
        print(f'Found in {slot} → {info['owner']}')
    else:
        print("Not found!")

# ----- STATUS ----- #
def status(self):
    for slot, info in self.data.items():
        status = "EMPTY" if info is None else f'{info['vehicle']}'
        print(f'{slot} → {status}')

# ----- ANALYTICS (PANDAS + VISUALIZATION) ----- #
def analytics(self):
    if not os.path.exists(CSV_FILE):
        print("No data available!")
        return

    df = pd.read_csv(CSV_FILE)

```

```

print("\nTotal Vehicles:", len(df))
print("Total Revenue:", df["Fee"].sum())

# Visualization
df["Fee"].plot(kind="hist")
plt.title("Parking Fee Distribution")
plt.show()

# ----- RESET ----- #
def reset(self):
    confirm = input("Reset? yes/no: ")
    if confirm.lower() == "yes":
        self.data = {}
        self.save_data()
        print("System reset!")

# ----- MENU ----- #
def menu(self):
    while True:
        print("\n--- SMART PARKING ---")
        print("1.Init 2.Park 3.Remove 4.Status 5.Search 6.Analytics 7.Reset 8.Exit")

        choice = input("Choice: ")

        try:
            if choice == "1":
                self.initialize()
            elif choice == "2":
                self.park()
            elif choice == "3":
                self.remove()
            elif choice == "4":
                self.status()
            elif choice == "5":
                self.search()
            elif choice == "6":
                self.analytics()
            elif choice == "7":
                self.reset()
            elif choice == "8":
                break
            else:
                print("Invalid choice!")

```

```
except Exception as e:  
    print("Error:", e)
```

```
#----- RUN ----- #  
if __name__ == "__main__":  
    system = ParkingSystem()  
    system.menu()
```


INTEGRATION AND SYSTEM TESTING OUTPUTS

Screen Shots:

```
===== SPAOAS MENU =====  
1. Initialize Parking  
2. Park Vehicle  
3. Remove Vehicle  
4. View Parking Status  
5. Search Vehicle  
6. View Analytics  
7. Reset System  
8. Exit  
Enter your choice: 2  
Parking Full!
```

Your Output

```
===== SPAOAS MENU =====  
1. Initialize Parking  
2. Park Vehicle  
3. Remove Vehicle  
4. View Parking Status  
5. Search Vehicle  
6. View Analytics  
7. Reset System  
8. Exit  
Enter your choice:
```

```
Enter your choice: 4
```

```
Parking Status:
```

```
SLOT_1 → EMPTY  
SLOT_2 → EMPTY  
SLOT_3 → EMPTY  
SLOT_4 → EMPTY  
SLOT_5 → EMPTY  
SLOT_6 → EMPTY  
SLOT_7 → EMPTY  
SLOT_8 → EMPTY  
SLOT_9 → EMPTY  
SLOT_10 → EMPTY  
SLOT_11 → EMPTY  
SLOT_12 → EMPTY  
SLOT_13 → EMPTY  
SLOT_14 → EMPTY
```

```
===== SPAOAS MENU =====
```

1. Initialize Parking
2. Park Vehicle
3. Remove Vehicle
4. View Parking Status
5. Search Vehicle
6. View Analytics
7. Reset System
8. Exit

```
Enter your choice: 5
```

```
Enter vehicle number to search: AP 39 BB 3941
```

```
Vehicle not found!
```

```
===== SPAOAS MENU =====  
1. Initialize Parking  
2. Park Vehicle  
3. Remove Vehicle  
4. View Parking Status  
5. Search Vehicle  
6. View Analytics  
7. Reset System  
8. Exit  
Enter your choice: 7  
Are you sure you want to reset all data? (yes/no): YES  
System reset complete.
```

```
===== SPAOAS MENU =====  
1. Initialize Parking  
2. Park Vehicle  
3. Remove Vehicle  
4. View Parking Status  
5. Search Vehicle  
6. View Analytics  
7. Reset System  
8. Exit  
Enter your choice: 8  
Exiting Smart Parking System...
```

CONCLUSION

This project on **Smart Parking Allocation and Occupancy** has been developed successfully to address the growing parking problem in modern urban environments. It provides a smart and organized method for managing parking space, detecting occupancy, and assigning slots efficiently. The system helps reduce the time spent searching for parking and improves the overall flow of vehicles in crowded areas.

The project is beneficial because it saves time, reduces confusion, improves space utilization, and makes parking management easier. It can be used in many public and private locations where proper parking control is needed. With future enhancements such as mobile app integration, cloud storage, automated billing, and sensor-based detection, the system can become even more advanced and practical. Overall, it is a useful and effective solution for modern parking needs.